Real-Time Route Planning and Online Order Dispatch for Bus-Booking Platforms*

Hao Zhou, Yucen Gao, Xiaofeng Gao**, and Guihai Chen

Shanghai Key Laboratory of Scalable Computing and Systems,
Department of Computer Science and Engineering,
Shanghai Jiao Tong University, China
{h-zhou,guo_ke}@sjtu.edu.cn, {gao-xf,g-chen}@cs.sjtu.edu.cn

Abstract. To cater to the high travel demands in Beijing Capital International Airport during 23:00-2:00, Beijing Traffic Management Bureau (BTMB) intends to develop a new service named bus-booking platforms. Compared to traditional airport shuttle buses, bus-booking platforms can conduct flexible route planning and online order dispatch while provide much lower price than the common car-hailing platform, e.g., Didi Chuxing. We conduct the real-time route planning by solving the standard Capacitated Vehicles Routing Problem based on order prediction. In addition, we focus on the design of the online order dispatch algorithm for bus-booking platforms, which is novel and extremely different from the traditional taxi order dispatch in car-hailing platforms. When an order is dispatched, multiple influence factors will be considered simultaneously, such as the bus capacity, the balanced distribution of the accepted orders and the travel time of passengers, all of which aim to provide a better user experience. Moreover, we prove that our online algorithms can achieve the tight competitive ratio in this paper, where the competitive ratio is the ratio between the solution of an online algorithm and the offline optimal solution.

Keywords: Order Dispatch · Online Algorithm · Intelligent Transportation System.

1 Introduction

The rapid deployment of transportation in recent years speeds up the travel between different cities and increases the travel frequency, which brings more traffic in return. To cater to the increasing travel demands and relieve the traffic pressure, some novel transportation concepts have been proposed and put into

* This work was supported by the National Key R&D Program of China [2018YFB1004703]; the National Natural Science Foundation of China [61872238, 61672353]; the Shanghai Science and Technology Fund [17510740200]; the Huawei Innovation Research Program [HO2018085286]; and the State Key Laboratory of Air Traffic Management System and Technology [SKLATM20180X].

** Corresponding author.

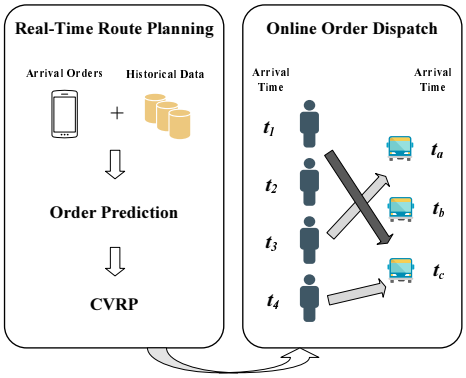

Fig. 1. The Framework of Bus Booking Platforms

the wide application, e.g., online car-hailing. However, some locations with high travel demands are not fully satisfied yet. According to the official statistics from Beijing Traffic Management Bureau (BTMB), the number of the passengers arrived at Beijing Capital International Airport is about 18,000 during midnight period (23:00-02:00). However, the local public transport capacity can only provide pick-up service for around 8,200 passengers, including 7,000 passengers by taxi and online car-hailing, and 1,200 passengers by airport shuttle buses (less than 7% of the total arrived passengers), leaving a large number of passengers waiting at the airport for a long duration. In order to provide a higher-quality service for passengers, BTMB intends to develop a new service named bus-booking platforms. Unlike car-hailing platforms, bus-booking platforms can dispatch several orders to one bus with much lower price. Compared to traditional airport shuttle buses, bus-booking platforms can conduct the real-time route planning and the flexible order dispatch, which contribute to a better user experience.

As is shown in Fig. 1, the general framework of bus-booking platforms consists of two component: Real-Time Route Planning and Online Order Dispatch. When a bus is available in the source station, bus-booking platforms compute the travel line for this bus by solving the standard Capacitated Vehicle Routing Problem (CVRP). Once a new order arrives, bus-booking platforms will reply to the passenger whether the order is accepted and which bus the order is dispatched to if it is accepted. Althought CVPR has been studied for many years, the previous works mainly concentrated on the offline cases where all the passengers and their destinations have been determined, which cannot directly apply to the real-time route planning [3, 2, 9]. In addition, the order dispatch problem for bus-booking platforms is novel and extremely different from the traditional taxi or car-hailing order dispatch problem, where the latter emphasizes on the matching of one order and one bus.

In this paper, we provide a complete overview for the bus-booking platform, from the framework design to the algorithm analysis. In order to overcome the problem where the common CVRP algorithm need to be given the destinations of passengers, we add the step of order prediction based on deep neural networks

(DNN). According to the arrived orders and historical data, bus-booking platforms will predict the basic attributes of new orders in next timeslot, containing the destination, the number of orders, etc. Besides the difference in the match pattern, most strategies for the order dispatch in car-hailing platforms are offline and heuristic, which cannot provide performance guarantee, e.g., the greedy mechanism in the early taxi-hailing platform [5], the offline Kuhn-Munkres (K-M) algorithm [7] and the hill-climbing method deployed in Didi Chuxing [13, 12]. Different from them, we propose two effective online order dispatch algotithms for bus-booking platforms. It has been proved that our online algorithms achieves the tight competitive ratio, where the competitive ratio is the ratio between the solution of an online algorithm and the offline optimal solution. When conducting the order dispatch, bus-booking platforms will consider several influence factors simultaneously, e.g., the bus capacity, the balanced distribution of the accepted orders, etc. In order to improve the user experience, the travel time of passengers is also involved in the bus-booking platforms. Notice that it is necessary for passengers to wait some time for the reply in [12, 13] since the platform has to make decisions after aggregating all the arrived orders in one timeslot. However, our online algorithms can reply to the passengers right now, which does not depend on next arrived orders in the same timeslot.

2 Real-Time Route Planning

- **Order Prediction.** We divide the whole time period to many timeslot $< t_1, t_2, \ldots, t_n >$. Suppose that d buses come to the source station in timeslot t_k. We predict the new orders which arrives in timeslot t_k and compute the route line of d buses based on them. In the early stage of bus-booking platforms, the station set is given and passengers select a certain station as their destination. Let $\mathcal{V}$ be the set of all the stations. Therefore, for each station $v \in \mathcal{V}$, we only need to predict the number of passengers who destined to this station in timeslot t_k in order to transform our problem to CVRP, which is denoted by $\hat{x}_{t_k,v}$. For convenience, we use the orders in the last h timeslots, i.e.,
$$\hat{x}_{t_k,v} = F_v(\hat{x}_{t_{k-h},v}, \ldots, \hat{x}_{t_{k-1},v}),\ v \in \mathcal{V}$$
Here a deep neural network is utilized to predict the function F_v.
- **CVRP.** Capacitated Vehicle Routing Problem (CVRP) is to compute the travel lines for the vehicles with capacity so as to satisfy the requests of all the passengers and minimize the travel cost of all the vehicles. When the number of predicted orders is small, CVRP can be solved to compute a Capacitated Minimum Spanning Tree (CMST) because all the passengers depart from the same source station. Therefore, any ρ-approximation algorithm for CMST is exactly an 2ρ-approximation algorithm for CVRP, where we transform each subtree in CMST to a tour by using the similar method in Travelling Salesman Problem [8]. If the number of orders is large, we have to reject some orders due to the limited number of buses. In this case, it is called Selective Vehicle Routing Problem and the genetic algorithms can be applied [2].

3 Online Order Dispatch

3.1 Model and Problem Formulation

Suppose that the orders arrive one by one over time in the source station, e.g., Beijing Capital International Airport. Each order is a quadruple (i, d_i, c_i, t_i), where i is the order number, d_i is the destination station, c_i is the number of passengers in this order and t_i is the arriving time. When an order arrives, there may be some available buses which are waiting to accept the orders again. Once a bus finishes its tour, it will return to the source station and wait the new orders. After an order arrives, we have to decide whether to accept this order and how to allocate it if it is accepted. Let x_{ij} denote whether the i-th order is dispatched to the j-th bus. Therefore, we have

$$\sum_{j \in B_i : d_i \in S_j} x_{ij} \leq 1, \ \forall i \in \mathcal{I} \tag{1}$$

where B_i is the set of the available buses in time t_i, S_j is the set of the stations that the j-th bus passes and $\mathcal{I}$ is the set of the orders. The passengers in the same order would like to take the same bus to the station. Let K_j be the number of seats in the j-th bus. Hence, the number of passengers allocated to the same bus should not exceed the capacity of this bus, i.e.,

$$\sum_{i \in I_j : d_i \in S_j} x_{ij} c_i \leq K_j, \ \forall j \in \mathcal{B} \tag{2}$$

where I_j denotes the set of the orders which arrive when the j-th bus are waiting in the source station and $\mathcal{B}$ is the set of all the available buses throughout the time period. Due to the limited number of buses, we cannot provide the pick-up service for all the passengers and have to reject some orders. Let $\mathcal{V}$ be the set of all the stations. In order to achieve the fairness, for each station $v \in \mathcal{V}$, there should be some orders which takes v as the destination to be accepted. In other words, the accepted orders should be balanced in all the stations instead of being centralized in a certain one. Therefore, we achieve this objective by using a bound R_v to limit the accepted orders or passengers destined to station v. The bound R_v can be determined based on the historical orders, the hot degree of each station, the total number of passengers which can be accepted by all the buses throughout the serving period, etc. Thus, we have

$$\sum_{i \in \mathcal{I} : d_i = v} \sum_{j \in B_i : v \in S_j} x_{ij} c_i \leq R_v, \ \forall v \in \mathcal{V} \tag{3}$$

Generally, the passengers would like to arrive their destinations as fast as possible and the user experience is influenced by the travel time, which should be considered in the allocation of orders. When an order arrives, there are several buses that can take the passenger to the destination station with different travel time. Let t_{ij} be the sum of the travel time of passengers in the i-th order by the

j-th bus. In order to decrease the average travel time of all the passengers and improve the user experience, we bound the sum of the travel time of passengers for each bus and for each station, i.e.,

$$\sum_{i \in I_j : d_i \in S_j} x_{ij} t_{ij} \leq T_j, \ \forall j \in \mathcal{B} \tag{4}$$

$$\sum_{i \in \mathcal{I} : d_i = v} \sum_{j \in B_i : v \in S_j} x_{ij} t_{ij} \leq T_v, \ \forall v \in \mathcal{V} \tag{5}$$

The bound T_j can be obtained according to the capacity of the bus K_j and the travel time of its tour path, while the bound T_v can be determined based on R_v and the average travel time from the source station to the destination station v. Each order is usually assigned with a priority p_i to represent its importance. For example, the order that is generated by a VIP user has a higher priority and should be accepted with higher probability. In addition, the rejected user is allowed to send the order request again and waits the reply. Therefore, the higher priority should be assigned to the order with longer waiting time. Our objective is to maximize the sum of the priority of all the accepted orders. Notice that if $p_i = 1$ for $\forall i \in \mathcal{I}$, the objective will be transformed to the maximization of the number of the accepted orders. Hence, we formulate our problem as a linear integer program as follows,

$$\begin{aligned} \max \quad & \sum_{i \in \mathcal{I}} \sum_{j \in B_i : d_i \in S_j} x_{ij} p_i, \\ \text{s.t.} \quad & \text{Constraint } (1), (2), (3), (4), (5) \\ & x_{ij} \in \{0, 1\}, \ \forall i \in \mathcal{I}, j \in \mathcal{B} \end{aligned} \tag{6}$$

3.2 Online Algorithm Design

Online Fractional Solution We first propose an online algorithm which produces a fractional solution and an integer solution can be obtained by using the randomized rounding technology. Before the further discussion, we present the dual of Program (6). We relax the integer constraint by replacing $x_{ij} \in \{0, 1\}$ with $x_{ij} \geq 0$ and take the dual variables for Constraint (1)-(5). Therefore, the dual objective is to minimize

$$\sum_{i \in \mathcal{I}} y_i + \sum_{j \in \mathcal{B}} (K_j z_j + T_j u_j) + \sum_{v \in \mathcal{V}} (R_v r_v + T_v q_v) \tag{7}$$

which subjects to

$$y_i + c_i(z_j + r_v) + t_{ij}(u_j + q_v) \geq p_i, \ \forall i \in \mathcal{I}, v = d_i, j \in B_i : d_i \in S_j \tag{8}$$

The Fractional Primal-Dual (FPD) algorithm is presented in Algorithm 1. All the dual variables are initialized as 0 in the beginning (lines 1-3). In the whole running process, Algorithm FPD ensures that the modified dual constraints

$$y_i + c_i(z_j + r_v) + t_{ij}(u_j + q_v) \geq p_i(1 - \exp(-\alpha)), \ \forall i \in \mathcal{I}, v = d_i, j \in B_i : d_i \in S_j$$

are always satisfied instead of Constraint (8), where α is an argument which is used to help the proof of Theorem 1 and is determined by the competitive ratio σ. Once an order arrives, Algorithm FPD finds a modified dual constraint that is violated (line 7) and updates the dual variables based on x_{ij} until this constraint is satisfied (lines 8-15). All the dual variables are some functions of the primal variables x_{ij} and keep undiminished during the updates. Therefore, the modified dual constraint always holds once it has been satisfied. Notice that the coefficient in the update formula (lines 11-15) is not increasing, e.g., p_{i_*}/c_{i_*}, $p_{i_\star}/c_{i_\star}$, $p_{i\_}/t_{i\_j}$, $p_{i_\sim}/t_{i_\sim j_\sim}$. Hence, the value of all the dual variables will not jump up during the updates, which will help our analysis for the competitive ratio. The argument β is similar to α in Algorithm FPD (lines 11-15), which will be determined by the competitive ratio. The loop will terminate when no modified dual constraint is violated. Therefore, Algorithm FPD produces a feasible fractional solution for the dual program with the modified dual constraints. In addition, Algorithm FPD also provides a fractional solution for the primal program, which is taken as the output (line 18).

Algorithm 1 Fractional Primal-Dual (FPD) Algorithm

Input: The bus set $\mathcal{B}$ and the station set $\mathcal{V}$.
Output: The fractional solution $\{x_{ij}\}_{i\in\mathcal{I},j\in\mathcal{B}}$.

1: **for** $i \in \mathcal{I}, j \in \mathcal{B}, v \in \mathcal{V}$ **do**
2: $\quad x_{ij} \leftarrow 0,\ y_i \leftarrow 0,\ z_j \leftarrow 0,\ u_j \leftarrow 0,\ r_v \leftarrow 0,\ q_v \leftarrow 0.$
3: **end for**
4: $I_a \leftarrow \emptyset$. /* the set of orders which have arrived */
5: **while** order (i, d_i, c_i, t_i) arrives **do**
6: $\quad I_a \leftarrow I_a \cup \{i\}$.
7: $\quad$ **while** $\exists j,\ y_i + c_i(z_j + r_v) + t_{ij}(u_j + q_v) < p_i(1 - \exp(-\alpha))$ **do**
8: $\quad\quad$ increase x_{ij} continuously.
9: $\quad\quad y_i \leftarrow p_i \exp\left[\alpha\left(\sum_{j\in B_i:d_i\in S_j} x_{ij} - 1\right)\right] - p_i \exp(-\alpha)$.
10: $\quad\quad i_* \leftarrow \arg\min_{i\in I_a\cap I_j:d_i\in S_j} \dfrac{p_i}{c_i},\quad i_\star \leftarrow \arg\min_{i\in I_a:d_i=v} \dfrac{p_i}{c_i}$.
11: $\quad\quad z_j \leftarrow \max\left(\dfrac{p_{i_*}}{c_{i_*}}\left[\exp\left(\beta\dfrac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{2K_j}\right) - 1\right], z_j\right)$.
12: $\quad\quad r_v \leftarrow \max\left(\dfrac{p_{i_\star}}{c_{i_\star}}\left[\exp\left(\beta\dfrac{\sum_{i\in\mathcal{I}:d_i=v}\sum_{j\in B_i:d_i\in S_j} x_{ij}c_i}{2R_v}\right) - 1\right], r_v\right)$.
13: $\quad\quad i_\_ \leftarrow \arg\min_{i\in I_a\cap I_j:d_i\in S_j} \dfrac{p_i}{t_{ij}},\quad (i_\sim, j_\sim) \leftarrow \arg\min_{i\in I_a, j\in B_i:d_i=v\in S_j} \dfrac{p_i}{t_{ij}}$.
14: $\quad\quad u_j \leftarrow \max\left(\dfrac{p_{i\_}}{t_{i\_j}}\left[\exp\left(\beta\dfrac{\sum_{i\in I_j:d_i\in S_j} x_{ij}t_{ij}}{2T_j}\right) - 1\right], u_j\right)$.
15: $\quad\quad q_v \leftarrow \max\left(\dfrac{p_{i_\sim}}{t_{i_\sim j_\sim}}\left[\exp\left(\beta\dfrac{\sum_{i\in\mathcal{I}:d_i=v}\sum_{j\in B_i:d_i\in S_j} x_{ij}t_{ij}}{2T_v}\right) - 1\right], q_v\right)$.
16: $\quad$ **end while**
17: **end while**
18: **return** $\{x_{ij}\}_{i\in\mathcal{I},j\in\mathcal{B}}$.

Theorem 1. *For any $0 < \sigma < 1$, Algorithm FPD produces an online fractional solution for the primal program, which achieves an σ-competitive ratio by setting $\alpha = -\ln\sigma$ and $\beta = \frac{1}{8}\ln^2\sigma$, but violates Constraint* (2) *for the j-th bus by at most $O(\log \Gamma_j / \log^2\sigma)$, Constraint* (3) *for station v by at most $O(\log \Lambda_v / \log^2\sigma)$, Constraint* (4) *for the j-th bus by at most $O(\log \Upsilon_j / \log^2\sigma)$ and Constraint* (5) *for station v by at most $O(\log \Psi_v / \log^2\sigma)$, where*

$$\Gamma_j = \max_{i\in I_j : d_i \in S_j}\{\frac{p_i}{c_i}\} \times \max_{i\in I_j : d_i \in S_j}\{\frac{c_i}{p_i}\}, \quad \Lambda_v = \max_{i\in \mathcal{I} : d_i = v}\{\frac{p_i}{c_i}\} \times \max_{i\in \mathcal{I} : d_i = v}\{\frac{c_i}{p_i}\}$$

$$\Upsilon_j = \max_{i\in I_j : d_i \in S_j}\{\frac{p_i}{t_{ij}}\} \times \max_{i\in I_j : d_i \in S_j}\{\frac{t_{ij}}{p_i}\}$$

$$\Psi_v = \max_{i\in \mathcal{I}, j\in B_i : d_i = v \in S_j}\{\frac{p_i}{t_{ij}}\} \times \max_{i\in \mathcal{I}, j\in B_i : d_i = v \in S_j}\{\frac{t_{ij}}{p_i}\}$$

Proof. See Appendix A for the detailed proof.

Fix σ to a constant, e.g., $1/2$, and let $\Sigma = \max_{j,v}\{\Gamma_j, \Lambda_v, \Upsilon_j, \Psi_v\}$. After Dividing K_j, R_v, T_j, T_v by $\log\Gamma_j, \log\Lambda_v, \log\Upsilon_j, \log\Psi_v$ for Constraints (2)-(5) in the primal program and running Algorithm FPD again, we can obtain a feasible fractional solution for the primal program which achieves an $O(\log\Sigma)$-competitive ratio and does not violate any constraint.

Improved Primal-Dual Algorithm Although an online integer solution can be obtained by using the randomized rounding technology for the fractional solution computed by Algorithm FPD, it only achieves an $O(\log\Sigma\log N)$-competitive ratio for the primal program, where $N = |\mathcal{I}|$ is the number of orders. Next we propose an improved primal-dual algorithm, which always produces an integer solution.

As is shown in Algorithm 2, all the dual variables are initialized as 0 in the beginning (lines 1-3). Once a new order arrives, Algorithm IPD determines whether to accept this order based on the dual constraints. If the dual constraints for this order have been satisfied, Algorithm IPD directly rejects it and does not update any dual variables (lines 5-7). Otherwise, Algorithm IPD finds the j'-th bus that satisfies $j' = \arg\min_{j\in B_i : d_i \in S_j} c_i z_j + t_{ij}(u_j + q_v)$ (line 5) and sets $y_i \leftarrow 1 - c_i(z_{j'} + r_v) - t_{ij^*}(v_{j'} + q_v)$ (line 10), where the latter ensures that all the dual constraints for this order are satisfied again. Meanwhile, Algorithm IPD dispatches this new order to the j'-th bus (line 9) and updates all the dual variables based on the update formulas (lines 11-14). Similar to Algorithm FPD, the argument ϵ in the update formulas is a constant that is determined by the competitive ratio.

Theorem 2. *For any constant $\epsilon > 0$, Algorithm IPD produces an online integer solution for the primal program, which achieves a $(1-\epsilon)$-competitive ratio, but violates Constraint* (2) *for the j-th bus by at most $O(\log\Gamma_j + \log 1/\epsilon)$, Constraint* (3) *for station v by at most $O(\log\Lambda_v + \log 1/\epsilon)$, Constraint* (4) *for the j-th*

Algorithm 2 Improved Primal-Dual (IPD) Algorithm

Input: The bus set $\mathcal{B}$ and the station set $\mathcal{V}$.
Output: The integer solution $\{x_{ij}\}_{i\in\mathcal{I},j\in\mathcal{B}}$.

for $i \in \mathcal{I}, j \in \mathcal{B}, v \in \mathcal{V}$ **do**
  $x_{ij} \leftarrow 0,\ y_i \leftarrow 0,\ z_j \leftarrow 0,\ u_j \leftarrow 0,\ r_v \leftarrow 0,\ q_v \leftarrow 0.$
end for
while order (i, d_i, c_i, t_i) arrives **do**
  $j' \leftarrow \arg\min_{j\in B_i:d_i\in S_j} c_i z_j + t_{ij}(u_j + q_v)$.
  if $c_i(z_{j'} + r_v) + t_{ij'}(u_{j'} + q_v) \geq p_i$ **then**
    reject order (i, d_i, c_i, t_i).
  else
    $x_{ij'} \leftarrow 1$.
    $y_i \leftarrow p_i - c_i(z_{j'} + r_v) - t_{ij'}(v_{j'} + q_v)$.
    $z_{j'} \leftarrow z_{j'}\left(1 + \frac{c_i}{K_{j'}}\right) + \frac{p_i\epsilon}{4K_{j'}}$.
    $r_v \leftarrow r_v\left(1 + \frac{c_i}{R_v}\right) + \frac{p_i\epsilon}{4R_v}$.
    $u_{j'} \leftarrow u_{j'}\left(1 + \frac{t_{ij'}}{T_{j'}}\right) + \frac{p_i\epsilon}{4T_{j'}}$.
    $q_v \leftarrow q_v\left(1 + \frac{t_{ij'}}{T_v}\right) + \frac{p_i\epsilon}{4T_v}$.
  end if
end while
return $\{x_{ij}\}_{i\in\mathcal{I},j\in\mathcal{B}}$.

bus by at most $O(\log \Upsilon_j + \log 1/\epsilon)$ *and Constraint* (5) *for station* v *by at most* $O(\log \Psi_v + \log 1/\epsilon)$*, where*

$$\Gamma_j = \max_{i\in I_j:d_i\in S_j}\{\frac{p_i}{c_i}\} \times \max_{i\in I_j:d_i\in S_j}\{\frac{c_i}{p_i}\},\ \ \Lambda_v = \max_{i\in\mathcal{I}:d_i=v}\{\frac{p_i}{c_i}\} \times \max_{i\in\mathcal{I}:d_i=v}\{\frac{c_i}{p_i}\}$$

$$\Upsilon_j = \max_{i\in I_j:d_i\in S_j}\{\frac{p_i}{t_{ij}}\} \times \max_{i\in I_j:d_i\in S_j}\{\frac{t_{ij}}{p_i}\}$$

$$\Psi_v = \max_{i\in\mathcal{I},j\in B_i:d_i=v\in S_j}\{\frac{p_i}{t_{ij}}\} \times \max_{i\in\mathcal{I},j\in B_i:d_i=v\in S_j}\{\frac{t_{ij}}{p_i}\}$$

Proof. See Appendix B for the detailed proof.

3.3 The Lower Bound

Consider an extreme example where there are only one bus j and only one station v, and all the passengers would like to go to station v. Suppose that the number of passengers in each order and the travel time from the source station to station v are both one unit, i.e., $c_i = t_{ij} = 1,\ \forall i \in \mathcal{I}$. In addition, we set $K_j = R_v = T_j = T_v = K$ in this example. Suppose the orders arrive in batches, where each batch has K orders. For the i-th order, let $p_i = 1/(M - m + 1)$ where M is a constant and this order locates in the m-th batch, i.e., $(m-1)K < i \leq mK$. Based on the above assumptions, this example can be formulated as follows,

$$\max \quad \sum_{i\in\mathcal{I}} x_{ij}p_i, \tag{9}$$

$$\text{s.t.} \quad \sum_{i\in\mathcal{I}} x_{ij} \le K, \tag{9a}$$

$$x_{ij} \in \{0,1\},\ \forall i \in \mathcal{I} \tag{9b}$$

Notice that Constraint (2)-(5) in the primal program (6) are transformed to Constraint (9a) and Constraint (1) is equivalent to Constraint (9b) in this example. Even for this simple example, we claim that any online algorithm cannot provide a better performance guarantee.

Theorem 3. *For any $0 < \rho \le 1$, a ρ-competitive online algorithm must violate Constraint* (9a) *by at least $\rho \ln M$ in this example.*

Proof. Notice that the priority of an order is monotone increasing with the batch. When the m-th batch of orders arrives, the optimal solution is to accept all the orders in the m-th batch and reject all the former orders. Denote the optimal solution by $\mathrm{OPT}_{\mathcal{I}_m}$ where $\mathcal{I}_m$ is the first m batch of orders. Denote by $A^{\rho}_{\mathcal{I}_m}$ the solution of a ρ-competitive algorithm for $\mathcal{I}_m$. Therefore, we have

$$A^{\rho}_{\mathcal{I}_m} = \sum_{i\in\mathcal{I}_m} x_{ij}p_i \ge \rho\ \mathrm{OPT}_{\mathcal{I}_m} = \frac{\rho K}{M-m+1}$$

Accumulate the above equality from $m = 1$ to M and let $|\mathcal{I}| = MK$. We have

$$\sum_{m=1}^{M}\sum_{i\in\mathcal{I}_m} x_{ij}p_i \ge \sum_{m=1}^{M} \frac{\rho K}{M-m+1} = \rho K \ln M$$

Notice that

$$\begin{aligned}\sum_{m=1}^{M}\sum_{i\in\mathcal{I}_m} x_{ij}p_i = \sum_{m=1}^{M}\sum_{l=1}^{m}\sum_{k=1}^{K} \frac{x_{ij}}{M-l+1} &= \sum_{l=1}^{M}\sum_{m=l}^{M}\sum_{k=1}^{K} \frac{x_{ij}}{M-l+1} \\ &= \sum_{l=1}^{M}\sum_{k=1}^{K} x_{ij} = \sum_{i\in\mathcal{I}} x_{ij}\end{aligned}$$

Combining the above equalities, we finish the proof here.

According to Theorem 3, in order to obtain an online algorithm that does not violate any constraint, its competitive ratio ρ has to satisfy $\rho \ln M \le 1$, i.e., $\rho \le 1/\ln M$. Notice that $\Gamma_j = \Lambda_v = \Upsilon_j = \Psi_v = M$ in this example. Therefore, Algorithm IPD achieves the tight competitive ratio for the online order dispatch problem. In addition, the integer constraint (9b) is not used in the proof of Theorem 3, which means that the lower bound can also apply to the fractional solution of Program (9). Hence, Algorithm FPD also provides the tight performance guarantee in the competitive ratio for the fractional solution of the online order dispatch problem.

4 Experiment

4.1 Experimental Implementation

We conduct a comparative experiment to evaluate our online order dispatch algorithm. Top 30 hot stations are selected and the travel time between the stations are obtained according to their geographical distance. We randomly generate m buses and n orders during midnight period (23:00-02:00). Each bus arrives at Beijing Capital International Airport at random time point and depart from it after ten-minute waiting. The capacity of each bus is a parameter in our experiment. The travel line of each bus is computed by solving Capacitated Vehicle Routing Problem based on the arrived and predicted orders. The priority of each order is a random real number in $(0, 1]$. In order to provide a better user experience, four constraints have been considered in Program (6). The threshold value of the number of the accepted orders in one station is calculated by multiplying the total capacity of all the buses by the ratio between the number of orders destined to this station and the total number of all the orders. The threshold value of the travel time in one bus is calculated by multiplying the bus capacity by the average travel time of all the stations in this bus line. The threshold value of the travel time in one station is calculated based on the average travel time from the source station.

Based on the above statement, we make a detailed description of four compared algorithms involved in the experiment. *Random:* A random allocation algorithm. When an order arrives, a bus is randomly assigned to accept this order. We judge whether the above constraints can be satisfied after the bus accepts the order. If they are satisfied, the bus accepts this order and the state of the bus is updated, otherwise refuses. *Greedy:* A greedy algorithm based on the maximum-remaining-seat-first principle. When an order arrives, we find one bus with the most remaining seats that can take the passengers in this order to their destination station. We judge whether the above constraints can be satisfied after the bus accepts the order. If they are satisfied, the bus accepts the order and updates its state, otherwise refuses. *IPD:* The online order dispatch algorithm in this paper. *OPT:* The offline optimal solution by using Groubi 8.1.0 [1] to solve the linear integer programm (6).

4.2 Experiment Result and Anlysis

In the realistic situation, the total number of passengers included in the orders is often greater than the total capacity of all the buses due to the limited number of buses. Accordingly, we set different parameters and implement five experiments. The parameters in each experiment are shown in Table 1.

As is shown in Fig. 2, Fig. 3 and Fig. 4, both Random and Greedy do not perform well in the experiments. For the number of orders accepted, the number of passengers served and the priority of orders accepted, Random and Greedy only achieve less than 50% of the optimal solution compared with OPT. However, our online algorithm IPD can increase the results by more than 20% compared

Table 1. Parameters in five experiments

Experiment	#Buses	Bus Capacity	#Orders
1	30	30	1200
2	50	30	1200
3	50	40	1200
4	50	30	900
5	50	30	1500

with Random and Greedy. We only change the number of orders in Experiment 2, Experiment 4 and Experiment 5. According to Fig. 5, Fig. 6 and Fig. 7, we can find that the total number of orders during this period will not have a significant impact on the number of orders accepted, the number of passengers served and the priority of orders accepted if it is greater than the total bus capacity. Instead, the total bus capacity and the travel time constraint are the major influence factors when comparing the results in Fig. 2-Fig. 4 and Fig. 5-Fig. 7. Therefore, our online algorithm can provide the service for more passengers with almost the same total travel time compared with Random and Greedy. In other words, our algorithm can achieve lower average travel time. Although OPT performs much better in all the experiments, it requires too long running time and to be given all the orders in advance, which cannot apply to online situations.

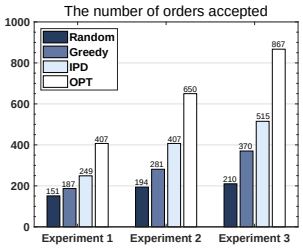

Fig. 2. Orders

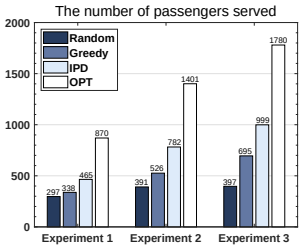

Fig. 3. Passengers

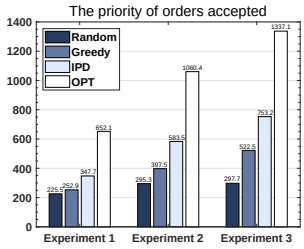

Fig. 4. Priority

5 Related Work

There are many works about the order dispatch problem for car-hailing platforms. Based on global positioning systems (GPS), the greedy mechanism was proposed for real-time taxi order dispatch where the order was matched with the nearest driver [5]. Although the above greedy strategy could significantly decrease the waiting time of passengers and improve the user experience, it did not consider the global profits of platforms and taxies. The queueing strategy based on the first-come-first-serve principle was also widely used in the early

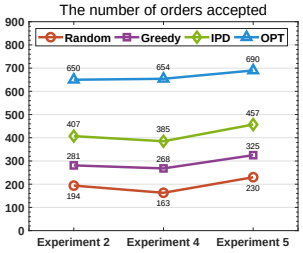

Fig. 5. Orders

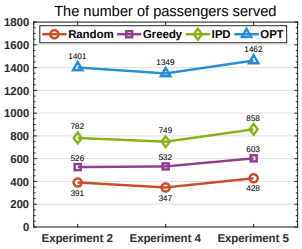

Fig. 6. Passengers

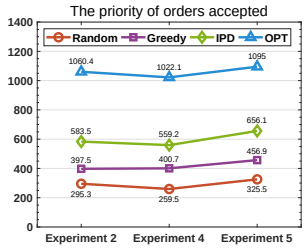

Fig. 7. Priority

taxi platforms [4, 10, 14]. The offline Kuhn-Munkres (KM) algorithm [7] was deployed in the Didi Chuxing platform, which collected all the arrived orders and the available cars in one timeslot and conducted the maximum bipartite matching [12]. In order to increase the total acceptance rate of orders, another work attempted to dispatch one order to several drivers, where the first one to accept the requirement obtained this order [13]. However, both of them focused on heuristic or offline algorithms, which could not provide the global performance guarantee. In addition, there were also some works that used the demand prediction to help order dispatch [6, 11], but all of them focused on taxi or car-hailing platforms.

6 Conclusion

In this paper, we presented a complete overview for a new service named bus-booking platforms. Real-time route planning in bus-booking platforms can be achieved by solving the standard CVRP based on order prediction. In addition, we proposed two novel online order dispatch algorithms to satisfy the real-time requirements from customers. We proved that our online algorithms can provide solid theoretical guarantee and touch the tight competitive ratio, where the competitive ratio is the ratio between the solution of an online algorithm and the offline optimal solution.

References

1. Gurobi. http://www.gurobi.com/.
2. M. Allahviranloo, J. Y. Chow, and W. W. Recker. Selective Vehicle Routing Problems under Uncertainty without Recourse. *Transportation Research Part E: Logistics and Transportation Review*, 62:68–88, 2014.
3. J. Cáceres-Cruz, P. Arias, D. Guimarans, D. Riera, and A. A. Juan. Rich Vehicle Routing Problem: Survey. *ACM Computing Surveys*, 47(2):32:1–32:28, 2014.
4. D.-H. Lee, H. Wang, R. L. Cheu, and S. H. Teo. Taxi Dispatch System based on Current Demands and Real-Time Traffic Conditions. *Transportation Research Record: Journal of the Transportation Research Board*, 1882:193–200, 2004.

5. Z. Liao. Real-Time Taxi Dispatching Using Global Positioning Systems. *Communications of the ACM (CACM)*, 46(5):81–83, 2003.
6. L. Moreira-Matias, J. Gama, M. Ferreira, J. Mendes-Moreira, and L. Damas. Predicting Taxi–Passenger Demand using Streaming Data. *IEEE Transactions on Intelligent Transportation Systems (TITS)*, 14(3):1393–1402, 2013.
7. J. Munkres. Algorithms for the Assignment and Transportation Problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.
8. C. H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, 1982.
9. T. K. Ralphs, L. Kopman, W. R. Pulleyblank, and L. E. Trotter. On the Capacitated Vehicle Routing Problem. *Mathematical Programming*, 94(2-3):343–359, 2003.
10. K. T. Seow, N. H. Dang, and D. Lee. A Collaborative Multiagent Taxi-Dispatch System. *IEEE Transactions on Automation Science and Engineering (TASAE)*, 7(3):607–616, 2010.
11. Y. Tong, Y. Chen, Z. Zhou, L. Chen, J. Wang, Q. Yang, J. Ye, and W. Lv. The Simpler The Better: A Unified Approach to Predicting Original Taxi Demands based on Large-Scale Online Platforms. In *ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 1653–1662, 2017.
12. Z. Xu, Z. Li, Q. Guan, D. Zhang, Q. Li, J. Nan, C. Liu, W. Bian, and J. Ye. Large-Scale Order Dispatch in On-Demand Ride-Hailing Platforms: A Learning and Planning Approach. In *ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 905–913, 2018.
13. L. Zhang, T. Hu, Y. Min, G. Wu, J. Zhang, P. Feng, P. Gong, and J. Ye. A Taxi Order Dispatch Model based on Combinatorial Optimization. In *ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 2151–2159, 2017.
14. R. Zhang and M. Pavone. Control of Robotic Mobility-on-Demand Systems: A Queueing-Theoretical Perspective. *The International Journal of Robotics Research*, 35(1-3):186–203, 2016.

A Proof of Theorem 1

Proof. Consider a modified primal program as follows and let $\mathrm{OPT}, \mathrm{OPT}_\alpha$ be the optimal objective value for the primal program (6) and the modified program (10), respectively. Obviously, we have $\mathrm{OPT}_\alpha \geq (1 - \exp(-\alpha))\mathrm{OPT}$.

$$\begin{aligned} \max \quad & \sum_{i \in \mathcal{I}} \sum_{j \in B_i : d_i \in S_j} x_{ij} p_i (1 - \exp(-\alpha)), && (10) \\ \text{s.t.} \quad & \text{Constraint } (1), (2), (3), (4), (5) \\ & x_{ij} \geq 0, \ \forall i \in \mathcal{I}, j \in \mathcal{B} \end{aligned}$$

Notice that Algorithm FPD exactly produces a feasible fractional solution for the dual of Program (10), where denote by D_α its objective value. Let P be the objective value computed by Algorithm FPD for the primal program (6). Therefore, we have $\dfrac{\partial P}{\partial x_{ij}} = p_i$ and

$$\frac{\partial D_\alpha}{\partial x_{ij}} = \frac{\partial y_i}{\partial x_{ij}} + K_j \frac{\partial z_j}{\partial x_{ij}} + T_j \frac{\partial u_j}{\partial x_{ij}} + R_v \frac{\partial r_v}{\partial x_{ij}} + T_v \frac{\partial q_v}{\partial x_{ij}} \quad (\text{Note: } v = d_i)$$

From line 9 in Algorithm FPD, we get

$$\frac{\partial y_i}{\partial x_{ij}} = \alpha p_i \exp\alpha \left(\sum_{j \in B_i : d_i \in S_j} x_{ij} - 1 \right)$$

Since x_{ij} increases continuously and the condition in line 7 in Algorithm FPD is satisfied, $y_i \leq p_i(1-\exp(-\alpha))$ is always guaranteed. Combining line 9, we have $\frac{\partial y_i}{\partial x_{ij}} \leq \alpha p_i$. Similarly, we can get

$$\frac{\partial z_j}{\partial x_{ij}} \leq \frac{\beta c_i}{2K_j} \frac{p_{i_*}}{c_{i_*}} \exp\beta \left(\frac{\sum_{i \in I_j : d_i \in S_j} x_{ij} c_i}{2K_j} \right)$$

Due to $c_i z_j \leq p_i(1-\exp(-\alpha)) \leq p_i$, we have

$$\frac{\partial z_j}{\partial x_{ij}} \leq \frac{\beta c_i}{2K_j} \left(\frac{p_i}{c_i} + \frac{p_{i_*}}{c_{i_*}} \right) \leq \frac{\beta c_i}{2K_j} 2 \frac{p_i}{c_i} = \frac{\beta p_i}{K_j}$$

By the same way, we can get $\frac{\partial r_v}{\partial x_{ij}} \leq \frac{\beta p_i}{R_v}$, $\frac{\partial u_j}{\partial x_{ij}} \leq \frac{\beta p_i}{T_j}$, $\frac{\partial q_v}{\partial x_{ij}} \leq \frac{\beta p_i}{T_v}$. Therefore, we have $\frac{\partial D_\alpha}{\partial x_{ij}} \leq (\alpha + 4\beta) p_i = (\alpha + 4\beta) \frac{\partial P}{\partial x_{ij}}$. From the weak duality property, we have $\mathrm{OPT}_\alpha \leq D_\alpha \leq (\alpha + 4\beta) P$. Therefore, we further get

$$P \geq \frac{\mathrm{OPT}_\alpha}{\alpha + 4\beta} \geq \frac{1 - \exp(-\alpha)}{\alpha + 4\beta} \mathrm{OPT}$$

Let $\sigma = (1-\exp(-\alpha))/(\alpha + 4\beta)$ and $\alpha = -\ln\sigma$. We can get

$$\beta = \frac{1 - \sigma + \sigma \ln \sigma}{4\sigma}$$

Let $g(\sigma) = 1 - \sigma + \sigma \ln \sigma - \frac{1}{2}\sigma \ln^2 \sigma$. Since $g'(\sigma) = -\frac{1}{2} \ln^2 \sigma \leq 0$ and $\sigma \in (0, 1]$, we have $g(\sigma) \geq g(1) = 0$. Therefore, we can get

$$\beta \geq \frac{\frac{1}{2}\sigma \ln^2 \sigma}{4\sigma} = \frac{1}{8} \ln^2 \sigma$$

Now consider all the constraints in the primal program (6). First Constraint (1) is always satisfied because $y_i \leq p_i(1-\exp(-\alpha))$ always holds. Let $i^* = \arg\max_{i \in I_j : d_i \in S_j} \frac{p_i}{c_i}$. Because z_j is increased continuously and Algorithm FPD will not increase z_j if the condition in line 7 is not satisfied, we have $z_j \leq \frac{p_{i^*}}{c_{i^*}}$, i.e.,

$$\frac{p_{i_*}}{c_{i_*}} \left[\exp\left(\beta \frac{\sum_{i \in I_j : d_i \in S_j} x_{ij} c_i}{2K_j} \right) - 1 \right] \leq \frac{p_{i^*}}{c_{i^*}}$$

Rearranging the above inequality, we have

$$\frac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{K_j} \leq \frac{2}{\beta}\log\left(\frac{p_{i^*}}{c_{i^*}}\frac{p_{i_*}}{c_{i_*}}+1\right) \leq \frac{16}{\ln^2\sigma}\log\left(\frac{p_{i^*}}{c_{i^*}}\frac{p_{i_*}}{c_{i_*}}+1\right) = O\left(\frac{\log\Gamma_j}{\log^2\sigma}\right)$$

By the same way, we finish the proof.

B Proof of Theorem 2

Proof. Let x and (y, z, r, u, q) be the primal and dual solution computed by Algorithm IPD while P and D be the objective values, respectively. We finish the proof of this theorem from the following two properties:

(1) The solution (y, z, r, u, q) is feasible for the dual program.
(2) When order (i, d_i, c_i, t_i) is accepted, the objective of the dual program increases $(1+\epsilon)p_i$.

For Property (1), if order (i, d_i, c_i, t_i) is rejected, we have $c_i(z_j + r_v) + t_{ij}(u_j + q_v) \geq p_i$ for $\forall j \in B_i : d_i \in S_j$. Otherwise, we set $y_i \leftarrow p_i - c_i(z_{j'} + r_v) - t_{ij'}(v_{j'} + q_v)$, which guarantees that $y_i + c_i(z_j + r_v) + t_{ij}(u_j + q_v) \geq p_i$ for $\forall j : d_i \in S_j$ because $j' \leftarrow \arg\min_{j\in B_i:d_i\in S_j} c_i z_j + t_{ij}(u_j + q_v)$.
For Property (2), when order (i, d_i, c_i, t_i) is accepted, all the dual variables will be updated. Therefore, the objective of the dual program increases

$$\begin{aligned}
\Delta D &= \Delta y_i + \Delta K_{j'} z_{j'} + \Delta R_v r_v + \Delta T_{j'} u_{j'} + \Delta T_v q_v \\
&= p_i - c_i(z_{j'} + r_v) - t_{ij'}(v_{j'} + q_v) + K_{j'}\left(\frac{z_{j'}c_i}{K_{j'}} + \frac{p_i\epsilon}{4K_{j'}}\right) + \\
&\quad R_v\left(\frac{r_v c_i}{R_v} + \frac{p_i\epsilon}{4R_v}\right) + T_{j'}\left(\frac{u_{j'}t_{ij'}}{T_{j'}} + \frac{p_i\epsilon}{4T_{j'}}\right) + T_v\left(\frac{q_v t_{ij'}}{T_v} + \frac{p_i\epsilon}{4T_v}\right) \\
&= (1+\epsilon)p_i
\end{aligned}$$

Based on Property (1)(2) and the weak duality, we get

$$P \geq \frac{D}{1+\epsilon} \geq \frac{\mathrm{OPT}}{1+\epsilon} \geq (1-\epsilon)\mathrm{OPT}$$

For the constraints, we consider the dual variables (z, r, u, q), respectively. Let z_j^t be the value of z_j after the t-th update. Let $i_* = \arg\min_{i:d_i\in S_j} \frac{p_i}{c_i}$. Therefore, we have

$$z_j^t = z_j^{t-1}\left(1 + \frac{c_i}{K_j}\right) + \frac{p_i\epsilon}{4K_j} \geq z_j^{t-1}\left(1 + \frac{c_i}{K_j}\right) + \frac{c_i\epsilon}{4K_j}\frac{p_{i_*}}{c_{i_*}}$$

Rearranging the above inequality, we can get

$$z_j^t + \frac{p_{i_*}\epsilon}{4c_{i_*}} \geq \left(z_j^{t-1} + \frac{p_{i_*}\epsilon}{4c_{i_*}}\right)\left(1 + \frac{c_i}{K_j}\right)$$

Let γ_j satisfy

$$\left(1+\frac{c_i}{K_j}\right) \geq \gamma_j^{\frac{c_i}{K_j}},\ \forall i : d_i \in S_j \tag{11}$$

Therefore, we have

$$\begin{aligned} z_j^t + \frac{p_{i_*}\epsilon}{4c_{i_*}} &\geq \left(z_j^{t-1} + \frac{p_{i_*}\epsilon}{4c_{i_*}}\right)\gamma_j^{\frac{c_i}{K_j}} \\ &\geq \left(z_j^0 + \frac{p_{i_*}\epsilon}{4c_{i_*}}\right)\gamma_j^{\frac{\sum_{i:d_i\in S_j} x_{ij}c_i}{K_j}} \\ &= \frac{p_{i_*}\epsilon}{4c_{i_*}}\gamma_j^{\frac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{K_j}} \end{aligned} \tag{12}$$

According to Algorithm IPD (lines 6-7), before the last update of $\widetilde{z_j}$, we have

$$c_i(z_j^{t-1} + q_v^{t-1}) + t_{ij}(u_j^{t-1} + q_v^{t-1}) < p_i$$

Hence, $z_j^{t-1} < \frac{p_i}{c_i}$. Let $i^* = \arg\max_{i\in B_j:d_i\in S_j} \frac{p_i}{c_i}$. After the last update, we can get

$$\begin{aligned} z_j^t < \frac{p_i}{c_i}\left(1+\frac{c_i}{K_j}\right) + \frac{p_i\epsilon}{4K_j} = \frac{p_i}{c_i} + \left(1+\frac{\epsilon}{4}\right)\frac{p_i}{K_j} &\leq \frac{p_i}{c_i} + \left(1+\frac{\epsilon}{4}\right)\frac{p_i}{c_i} \\ &\leq \frac{p_{i^*}}{c_{i^*}}\left(2+\frac{\epsilon}{4}\right) \end{aligned}$$

Combining Inequality (12), we have

$$\frac{p_{i^*}}{c_{i^*}}\left(2+\frac{\epsilon}{4}\right) \geq z_j^t \geq \frac{p_{i_*}\epsilon}{4c_{i_*}}\left(\gamma_j^{\frac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{K_j}} - 1\right)$$

Therefore,

$$\frac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{K_j} \leq \log_{\gamma_j}\left(\frac{p_{i^*}}{c_{i^*}}\frac{c_{i_*}}{p_{i_*}}\left(\frac{8}{\epsilon}+1\right)+1\right) \leq \log_{\gamma_j}\frac{p_{i'}}{c_{i'}}\frac{c_{i^*}}{p_{i^*}}\left(\frac{8}{\epsilon}+2\right)$$

Now consider γ_j. From Equality (11), we have

$$\ln\gamma_j \leq \min_{i\in I_j:d_i\in S_j} \frac{\ln(1+\frac{c_i}{K_j})}{\frac{c_i}{K_j}}$$

Because $0 \leq \frac{c_i}{K_j} \leq 1$ and $f(x) = \frac{\ln(1+x)}{x}$ is a monotone decreasing function, Equality (11) is satisfied when $\gamma_j = 2$. Therefore,

$$\frac{\sum_{i\in I_j:d_i\in S_j} x_{ij}c_i}{K_j} \leq \left(\log_2\frac{p_{i^*}}{c_{i^*}}\frac{c_{i_*}}{p_{i_*}}\left(\frac{8}{\epsilon}+2\right)\right) = O\left(\log\Gamma_j + \log\frac{1}{\epsilon}\right)$$

By the same way, we finish this proof.